Bank Management System

Problem Statement

To design and implement a **Bank Management System using Java** that allows users to create bank accounts, perform ATM operations such as deposit, withdrawal, money transfer, and view account statements. The system ensures basic security using PIN authentication and restricts access so that users can only view and manage their own accounts, while the admin can monitor all accounts.

Concepts Used

Concept	Purpose
Classes & Objects	Represent bank accounts and system operations
Methods	Perform operations like deposit, withdraw, transfer
Arrays	Store multiple account records
ArrayList	Maintain transaction history
Strings	Store names, account numbers, and transaction logs
Conditional Statements	Validate PIN, balance checks, and account access
Loops	Run menu system and iterate through accounts
Encapsulation	Protect account data like PIN and balance

Short Description of Working

1. The system starts with a **main menu** that allows users to create accounts or access banking services.
2. A new user must **create an account** by providing personal details and setting a secure PIN.
3. Existing users must **log in using account number and PIN** to perform operations.
4. Users can perform banking operations such as:
    - Deposit money
    - Withdraw money
    - Transfer money to another account
    - View account details and mini statement
5. Every transaction is recorded in a **transaction history list**.
6. An **admin panel** allows authorized staff to view all accounts in the system.
7. The program runs in a **console-based interactive menu system** until the user exits.

BankingApplication.java

```java
/**
 *Project: Simple Bank Management System.
 *      It allows users to create accounts, deposit or withdraw money,
 * transfer funds, and view their transactions.
 * Access is protected using a PIN, and an admin panel can view all accounts.
 * This project demonstrates basic OOP concepts, arrays, methods, and console-based menus.
**/

import java.util.*;

/* Account class stores all account information and transactions */
class Account {  19 usages

    int accountNumber;  5 usages
    String name;  2 usages
    String phone;  3 usages
    int pin;  2 usages
    double balance;  6 usages

    ArrayList<String> transactions = new ArrayList<>();  5 usages

    /* Constructor creates a new bank account */
    Account(int accountNumber, String name, String phone, int pin, double balance) {  1 usage
        this.accountNumber = accountNumber;
        this.name = name;
        this.phone = phone;
        this.pin = pin;
        this.balance = balance;

        transactions.add("Account created with balance: " + balance);
    }

    /* Deposit money into account */
    void deposit(double amount) {  2 usages
        balance += amount;
        transactions.add("Deposited: " + amount);
    }

```

```java
    /* Withdraw money if sufficient balance */
    boolean withdraw(double amount) {  2 usages
        if (amount > balance) {
            return false;
        }

        balance -= amount;
        transactions.add("Withdrawn: " + amount);
        return true;
    }

    /* Add transfer transaction record */
    void addTransaction(String message) {  2 usages
        transactions.add(message);
    }

    /* Print account details */
    void showDetails() {  2 usages
        System.out.println("\n===== ACCOUNT DETAILS =====");
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Name           : " + name);
        System.out.println("Phone          : " + phone);
        System.out.println("Balance        : " + balance);
    }

    /* Print account statement */
    void showStatement() {  2 usages
        System.out.println("\n===== MINI STATEMENT =====");

        for (String t : transactions) {
            System.out.println(t);
        }
    }
}

```

```java

/* Main banking system class which controls all operations */
class BankSystem {  12 usages

    static Scanner sc = new Scanner(System.in);  14 usages

    /* Array storing all bank accounts */
    static Account accounts[] = new Account[100];  4 usages
    static int accountCount = 0;  4 usages

    /* Admin credentials */
    static String adminUser = "admin";  1 usage
    static String adminPass = "bank123";  1 usage

    static int nextAccountNumber = 1001;  3 usages

    /* Method to create a new account */
    static void createAccount() {  1 usage

        System.out.println("\n===== CREATE ACCOUNT =====");

        System.out.print("Enter Name: ");
        String name = sc.next();

        System.out.print("Enter Phone: ");
        String phone = sc.next();

        System.out.print("Set 4 digit PIN: ");
        int pin = sc.nextInt();

        System.out.print("Initial Deposit: ");
        double balance = sc.nextDouble();

        Account acc = new Account(nextAccountNumber, name, phone, pin, balance);

        accounts[accountCount] = acc;
        accountCount++;

        System.out.println("\nAccount Created Successfully!");
        System.out.println("Your Account Number: " + nextAccountNumber);

        nextAccountNumber++;
    }

    /* Method to search account using account number */
    static Account findAccount(int accNo) {  2 usages

        for (int i = 0; i < accountCount; i++) {
            if (accounts[i].accountNumber == accNo)
                return accounts[i];
        }

        return null;
    }
```

```java

    /* User login using account number and pin */
    static Account login() {  6 usages

        System.out.print("Enter Account Number: ");
        int accNo = sc.nextInt();

        Account acc = findAccount(accNo);

        if (acc == null) {
            System.out.println("Account not found");
            return null;
        }

        System.out.print("Enter PIN: ");
        int pin = sc.nextInt();

        if (acc.pin != pin) {
            System.out.println("Incorrect PIN");
            return null;
        }

        return acc;
    }

    /* Deposit money to account */
    static void depositMoney(Account acc) {  1 usage

        System.out.print("Enter amount to deposit: ");
        double amt = sc.nextDouble();

        acc.deposit(amt);

        System.out.println("Deposit successful");
    }

    /* Withdraw money using ATM */
    static void withdrawMoney(Account acc) {  1 usage

        System.out.print("Enter withdrawal amount: ");
        double amt = sc.nextDouble();

        if (acc.withdraw(amt)) {
            System.out.println("Withdrawal successful");
        } else {
            System.out.println("Insufficient balance");
        }
    }

```

```java

    /* Transfer money between two accounts */
    static void transferMoney(Account sender) {  1 usage

        System.out.print("Enter receiver account number: ");
        int receiverNo = sc.nextInt();

        Account receiver = findAccount(receiverNo);

        if (receiver == null) {
            System.out.println("Receiver account not found");
            return;
        }

        System.out.print("Enter transfer amount: ");
        double amount = sc.nextDouble();

        if (sender.withdraw(amount)) {

            receiver.deposit(amount);

            sender.addTransaction("Transferred " + amount + " to " + receiver.accountNumber);
            receiver.addTransaction("Received " + amount + " from " + sender.accountNumber);

            System.out.println("Transfer successful");
        } else {
            System.out.println("Insufficient balance");
        }
    }

    /* Update user details */
@   static void updateAccount(Account acc) {  1 usage

        System.out.println("\n===== UPDATE ACCOUNT =====");

        System.out.print("Enter new phone number: ");
        acc.phone = sc.next();

        System.out.println("Phone updated successfully");
    }

```

```java
    /* ATM menu for user */
    static void atmMenu(Account acc) {  1 usage

        int choice;

        do {
            System.out.println("\n===== ATM MENU =====");

            System.out.println("1 Withdraw");
            System.out.println("2 Check Balance");
            System.out.println("3 Mini Statement");
            System.out.println("4 Exit ATM");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    withdrawMoney(acc);
                    break;
                case 2:
                    System.out.println("Balance: " + acc.balance);
                    break;
                case 3:
                    acc.showStatement();
                    break;
            }

        } while (choice != 4);

    }
```

```java

    /* Admin login to see all accounts */
    static void adminPanel() {  1 usage

        System.out.print("Admin Username: ");
        String user = sc.next();

        System.out.print("Admin Password: ");
        String pass = sc.next();

        if (user.equals(adminUser) && pass.equals(adminPass)) {
            System.out.println("\n===== ALL ACCOUNTS =====");

            for (int i = 0; i < accountCount; i++) {
                accounts[i].showDetails();
            }
        } else {
            System.out.println("Invalid admin login");
        }
    }
}
```

```java

/* Main class containing main menu */
public class BankingApplication {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int choice;

        while (true) {

            System.out.println("\n====== BANK MANAGEMENT SYSTEM ======");

            System.out.println("1 Create Account");
            System.out.println("2 View My Account");
            System.out.println("3 ATM System");
            System.out.println("4 Deposit Money");
            System.out.println("5 Transfer Money");
            System.out.println("6 Update Account");
            System.out.println("7 View Statement");
            System.out.println("8 Admin Panel");
            System.out.println("9 Exit");

            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    BankSystem.createAccount();
                    break;
                case 2: {
                    Account acc = BankSystem.login();
                    if (acc != null)
                        acc.showDetails();
                    break;
                }
                case 3: {
                    Account acc = BankSystem.login();
                    if (acc != null)
                        BankSystem.atmMenu(acc);
                    break;
```

```
                }
                case 4: {
                    Account acc = BankSystem.login();
                    if (acc != null)
                        BankSystem.depositMoney(acc);
                    break;
                }
                case 5: {
                    Account acc = BankSystem.login();
                    if (acc != null)
                        BankSystem.transferMoney(acc);
                    break;
                }
                case 6: {
                    Account acc = BankSystem.login();
                    if (acc != null)
                        BankSystem.updateAccount(acc);
                    break;
                }
                case 7: {
                    Account acc = BankSystem.login();
                    if (acc != null)
                        acc.showStatement();
                    break;
                }
                case 8:
                    BankSystem.adminPanel();
                    break;
                case 9:
                    System.out.println("Thank you for using the bank system");
                    System.exit( status: 0);
                default:
                    System.out.println(("Invalid choice! Please choose between 1-9."));
                    break;
            }
        }
    }
}
```